



Hugging Face гораздо веселее играть с друзьями и коллегами! 🤗 [Вступить в организацию](#)

Отклоните это сообщение

← Вернуться к статьям

G2P сокращает количество речевых моделей



Статья сообщества

Опубликовано 5 февраля 2025 г.

▲ Голосуйте 97



Шестиградие

[шестиградие](#) Подписаться

🔗 Предыстория

Графемы — это единицы системы письменности (например, буквы в английском языке). **Фонемы** — это отдельные единицы звука.

G2P — это проблема преобразования графем в фонемы, часто в виде отношения «многие ко многим», характерного для конкретного языка. Одна последовательность графем может соответствовать нескольким последовательностям фонем:

- Можешь *перечитать* это для меня: reread => /ɪˈɪd/ (пере-ЧИТАТЬ)
- Да, я *перечитал* это для тебя: reread => /ɪˈɪəd/ (пере-ЧИТАТЬ)
- Давай еще раз *перечитаем*: reread => /ɪˈɪd/ (пере-ЧИТАТЬ)

А несколько цепочек графем могут соответствовать одной цепочке фонем, например cereal, serial и ate, eight.

Для целей данного обсуждения: текстовые модели генерируют текст, модели изображений — изображения, модели видео — видео, а аудиомодели — аудио. Речевые модели являются подтипом аудиомоделей:

```
generative_models/  
├── text/  
│   ├── GPT  
│   └── ...  
├── image/  
├── video/  
├── ...  
└── audio/  
    ├── speech/  
    │   ├── Kokoro-82M  
    │   └── ...  
    ├── music/  
    ├── sfx/  
    └── ...
```

🔗 Гипотеза сжатия G2P

Гипотеза: предварительная обработка входных данных G2P позволяет уменьшить размер речевых моделей, как самих моделей, так и наборов данных. Другими словами, если перед передачей текста в модель синтеза речи применить G2P к входным данным, можно добиться сопоставимого рейтинга Elo с меньшим количеством параметров и данных. Последнее является следствием, поскольку хорошо известно, что более крупные модели требуют больше данных.

Это понимание основано на моих собственных эмпирических наблюдениях и опыте — у меня нет таблиц абляции, которые я мог бы вам показать. Скорее всего, это не новое наблюдение, и для многих исследователей оно может быть очевидным.

Чем ниже энтропия ваших данных, тем меньше может быть ваша модель, при этом сохраняя целевую производительность. Даже крошечный классификатор цифр может обеспечить точность 99 % на наборе данных MNIST. Аналогичный классификатор изображений кошек и собак, где кошки и собаки могут быть повернуты в любом направлении, должен быть гораздо больше.

Использование фонем вместо исходного текста в качестве входных данных существенно снижает входную энтропию задачи преобразования текста в речь.

🔗 **Тяжеловесы: миллиарды параметров**

Речевые модели, которые выполняют сквозную обработку графем и аудио, неявно отводят часть своих параметров для преобразования графем в речь. Часто такие модели предсказывают скрытые аудиотокены на основе изученных кодовых книг, имеют относительно большое количество параметров и соответствующие обучающие наборы данных.

Parakeet

For example, Parakeet described in mid-2024 by Darefsky et al trains "a 3B parameter encoder-decoder transformer model" on "a ~100,000 hour dataset of audio-transcription pairs" and consequently exhibits exceptional naturalness, along with the ability to laugh, cough, etc. Parakeet predicts latent tokens "conditioned on raw text" and thus does not involve explicit G2P preprocessing. Two asides:

- In machine learning, you get what you pay for, and a model's parameter count is (in many ways) its price.
- Since Parakeet was trained on podcasts, some of its samples sound similar in nature to NotebookLM. This is pure speculation, but it would not surprise me if NotebookLM uses a similar transformer+codebooks approach, although I assume at a smaller size for cost effectiveness. Neither Parakeet nor NotebookLM would be the first to use such an approach.

Llasa

“Llasa is a text-to-speech (TTS) system that extends the text-based LLaMA (1B, 3B, and 8B) language model by incorporating speech tokens from the XCodec2 codebook, which contains 65,536 tokens. Llasa was trained on a dataset comprising 250,000 hours of Chinese-English speech data. The model is capable of generating speech either solely from input text or by utilizing a given speech prompt.

<https://huggingface.co/HKUST-Audio/Llasa-3B>”

Compared to those 65k tokens in XCodec2's codebook, you might only have on the order of dozens of phonemes after G2P preprocessing.

🔗 Featherweights: Millions of params

So if transformers can solve G2P neurally *in* the model, why bother with G2P preprocessing on the input? As stated in the hypothesis: model compression.

If Parakeet (3B) represents the heavyweight class of TTS models, Piper is in the featherweight class. A Piper model sits between 5M to 32M params, uses a VITS architecture, and takes espeak-ng phonemes as input to rapidly generate speech, albeit at the relatively low quality you would expect from such a small model. The parameter counts of Parakeet and Piper models are separated by 2-3 OOMs, or 100-1000x.

🔗 G2P Flavors: Lookup, Rules, and Neural

G2P is among the factors that enable (potentially lossy) compression from heavyweight to featherweight speech models. Notable G2P categories include:

- Pronunciation dictionaries like CMUdict are simple but brittle. If your dictionary is perfect and pronunciation is context-independent, any dictionary hit should be golden, but a dictionary miss or context-dependent pronunciation demands a more sophisticated G2P solution.

- espeak-ng is a rule-based G2P engine whose README states: "Compact size. The program and its data, including many languages, totals about few Mbytes." Rule-based G2P is still fast, but can fail due to insufficient rule coverage (e.g. missing regex for currency or time), or when there are exceptions to the rules (rare/foreign words).
- Neural G2P should in theory generalize far better than either of the above, but usually costs more compute to run, and like other transformer-with-softmax applications, could hallucinate in opaque ways.

I am currently rolling a hybrid G2P solution named Misaki which starts with lookup tables and some very basic rules for English. OOD words fall back to a configurable dealer's choice, which might be a rules-based system like espeak-ng, or a neural seq2seq model. In select situations, like English homographs `axes` `bass` `bow` `lead` `tear`, the G2P should be escalated up to neural nets (still a TODO). The hope is that such a hybrid solution achieves a good balance of speed and performance, while also being flexible and interpretable. Under this regime, G2P failures are less inexplicable hallucinations of some black box, and more "it just wasn't in the dictionary."

🔗 G2P is not a free lunch

Like the infamous strawberry tokenization blindspot, if you do explicit G2P preprocessing, then any upstream G2P nuances and errors will have negative downstream impacts that your speech model may be unable to recover from. Also, unless you can find an effective G2P scheme for things like laughs/coughs/sighs, or side-channel another way of modeling these sounds (diffusion, probably), it is unlikely a pure G2P-based speech model can pull these off as expressively and effectively as an end-to-end speech model.

If your G2P is neural, you have to add the time and compute cost of that neural G2P in front of your speech model, which adds latency.

And thus far, I have found G2P to be a per-language endeavor. Having a G2P engine for English does not automatically mean you can G2P Chinese as well.

🔗 Footnote

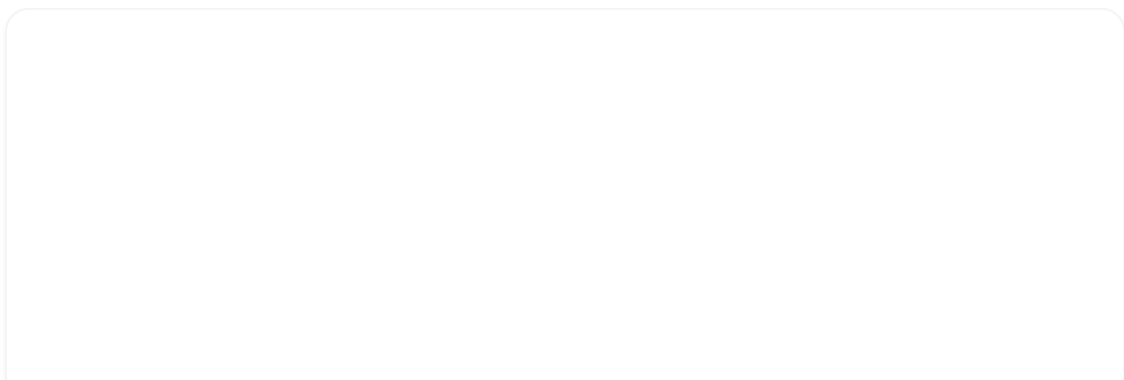
This post lacks some of the rigor you'd see in an academic paper, but it is grounded in experience. Its entirely possible that the G2P hypothesis will fall to the bitter lesson. And even if it doesn't, improved hardware and new architectures could mean that if people can run today's B-param speech models on tomorrow's mobile phones at near zero latency, then eventually no one will practically care about M-param speech models, in the same way no one practically cares about K-param LMs beyond education.

But "eventually" isn't here yet. I suspect M-param speech models should remain relevant for at least the near future.

This article is one potential answer to the question "How is Kokoro TTS so good with so few parameters?"



More from this author



Upgrading Kokoro: natural TTS for short bursts



△ 33 November 23, 2024

👏 Community

🌺 **Million** Feb 12, 2025



A very good explanation!



↩ Reply

🌐 **mahwizzzz** Mar 3, 2025



it was so good, nice work



↩ Reply

🇺🇸 **nam194** Apr 3, 2025



thanks for sharing



↩ Reply

👤 **m-aliabbas1** Apr 21, 2025



Solid Word toward making efficient text-to-speech model for low resource languages.



↩ Reply

🌺 **mayowadan** Jun 25, 2025



Love this explainer

Way more approachable than an academic paper

Noticed an issue with “live” in particular so I look forward to the neural augmentation for homonyms you’ve mentioned.

Let me know if you’d like any help with it whatsoever



↩ Reply

 **neurlang** Aug 14, 2025



Solid article from a TextToSpeech guru!



↩ Reply

 **TomSchelsen** Oct 15, 2025 • edited Oct 15, 2025



I don't know if the best place is here or on Github but here we go :

- what you describe you want to do seems to directly relate to <https://github.com/xinjli/transphone> and <https://github.com/dmort27/epitran> , are you aware of those ? I am not against a more up-to-date library since those don't seem maintained, but maybe there is some things to borrow.
- one interesting next step is to not stop at phonemes but go down to articulatory features, see https://github.com/DigitalPhonetics/IMS-Toucan/blob/StochasticProsodyModeling/Preprocessing/articulatory_features.py / <https://github.com/DigitalPhonetics/IMS-Toucan/blob/StochasticProsodyModeling/Preprocessing/TextFrontend.py>.

The main actual problem with G2P/text processing for me currently is that I see too often TTS papers' code be like : "I'll cover English by copy-pasting text preprocessing code from Tacotron (including transforming "Mr." into "mister", etc...), add my own language, and stop there". Having a goto, really massively multilingual text processing library that doesn't need to pull in espeak, would allow researchers to focus on the modeling part, and avoid the rest of us having to systematically fork the code to cover languages other than English/Chinese.



↩ Reply

Edit

Preview

Start discussing this article

 Tap or paste here to upload images

 Comment

 System theme

Company

TOS

Privacy

About

Careers

Website

Models

Datasets

Spaces

Pricing

Docs

